

Implementing set

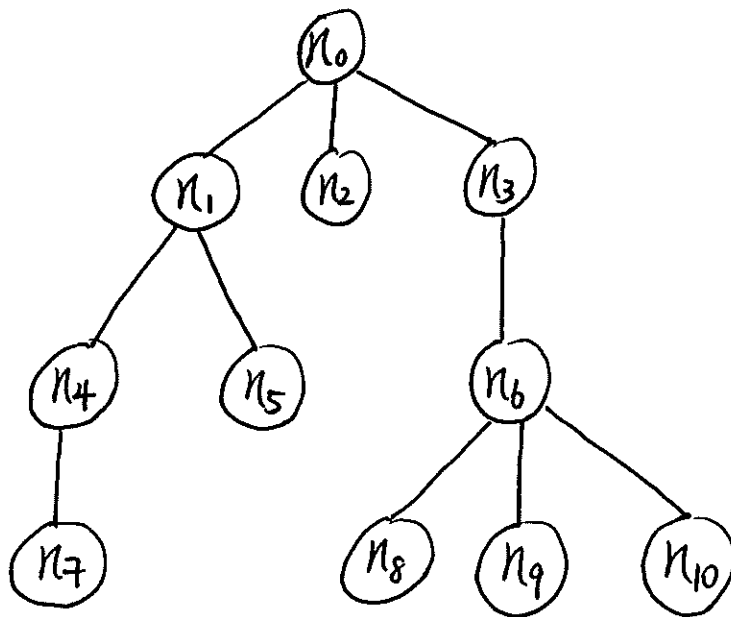
set is an associative container based on ^{Binary} search trees.

* Tree : non-linear data structure

Definition : Tree is a set of nodes storing elements such that each node has a parent-child relationship that satisfies the following properties

- ① Empty tree has nothing (No Node)
- ② If T is not an Empty Tree, it has a unique node, called the root of T , which has no parent.
- ③ Each node in T other than the root, has a unique parent node, and has 0 or more children.

e.g.



Which one is the root?

- * Every node with parent w is a child of w .
- * If a node has no child, we call it a leaf.

How many leaves?

- * external nodes : leaves
- * internal nodes : non-leaves How many internal nodes?
- * Ancestor: a is an ancestor of b if $a=b$ or
 a is an ancestor of the parent of b .

This is a recursive definition.

Review Recursion

- // two tree nodes a and b
- // You have access to the root, $\text{parent}()$
- // A recursive method returns true if a is an ancestor of b
- // false otherwise.

```
bool isAncestor(Node a, Node b) {
```

```
    // 1. a simple base case (contains no recursion)
```

```
    if (a == b) return true;
```

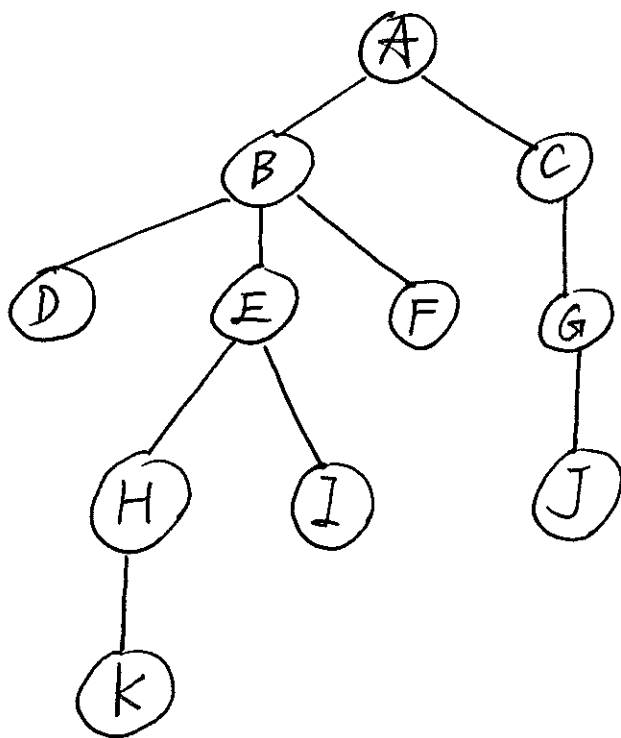
```
    if (b == root) return false;
```

```
    // 2. a set of rules reduces all other cases toward the base
```

```
    return isAncestor(a, b.parent());
```

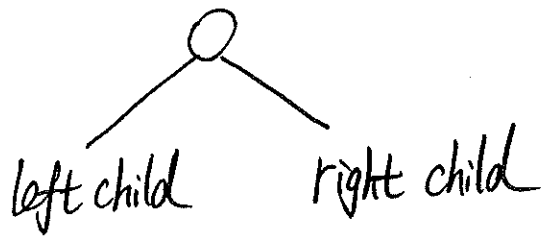
```
}
```

- * Descendent : a is a descendent of b if and only if b is an ancestor of a .
- * Subtree of T : For any node a in a tree T , the tree consisting of all descendants of a is a subtree of T .
root of the subtree?
- * edge : a line between a node and its parent.
- * Depth of a node a : the number of edges from a to the root
- * Height of a node a : the number of edges on the longest path from a to a leaf
- * Height of a tree = Height of its root
- * Height of an empty tree = -1



Binary Tree:

Every node has at most two children

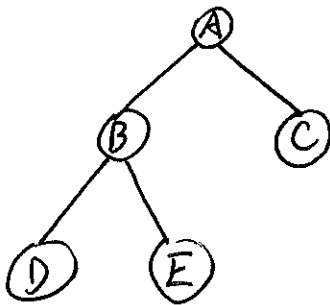


e.g.



is this a binary tree?

e.g.



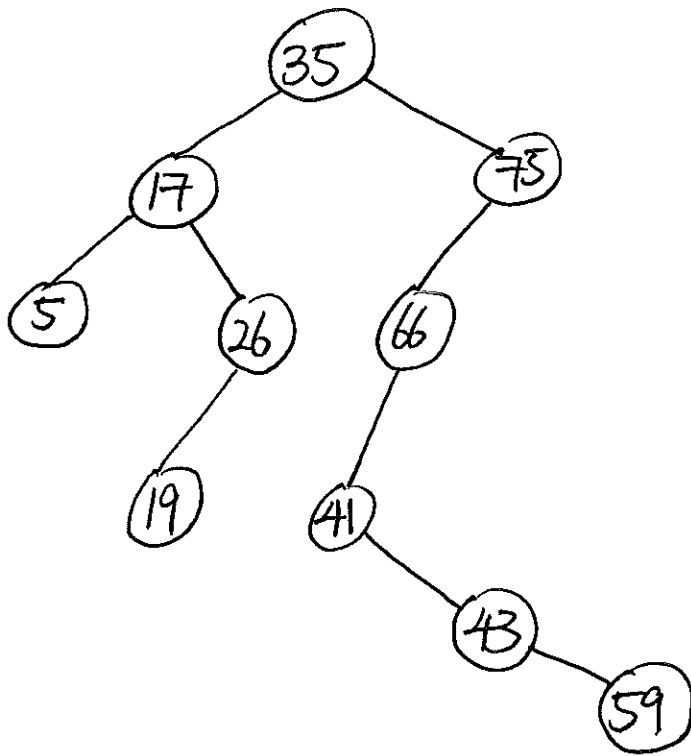
Binary search Tree: is a binary tree such that each node stores a $\langle \text{key}, \text{value} \rangle$ pair and for any node a in a BST,

keys in the left subtree of a $<$ key of a

keys in the right subtree of a $>$ key of a

e.g. Build a BST to store the following keys

35, 17, 26, 19, 75, 66, 41, 5, 43, 59



e.g. 40, 15, 63, 69, 52, 7, 34

insert (40, root)

insert (15, root)



insert (15, root → left)

insert (63, root)



insert (63, root → right)

insert (69, root)



insert (69, 63)



insert (69, 63 → right)

insert (52, root)

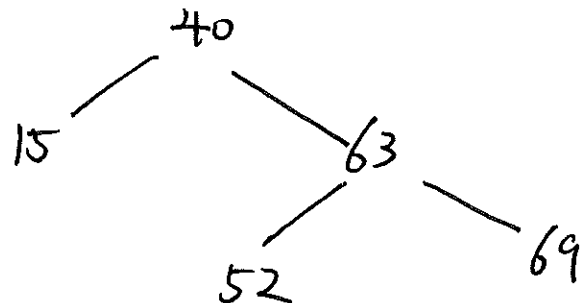
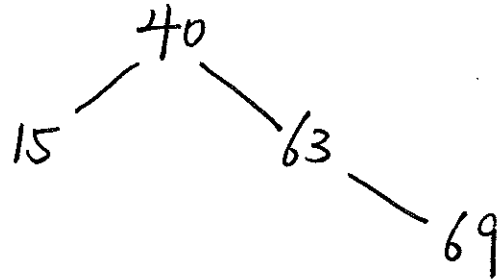
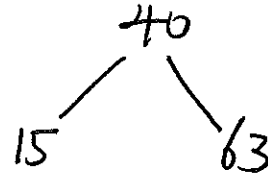
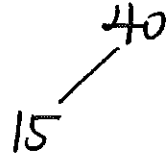


insert (52, 63)



insert (52, 63 → left)

40



Node* insert (T val, Node* t, Node* parent)

```
{  
    if (t == nullptr) {  
        ++ m_size;  
        t = new Node(val, nullptr, nullptr, parent);  
        return t;  
    }
```

```
}
```

```
if (val < t->data)
```

```
{  
    return insert (val, t->left, t);
```

```
} else if (val > t->data) {
```

```
    return insert (val, t->right, t);
```

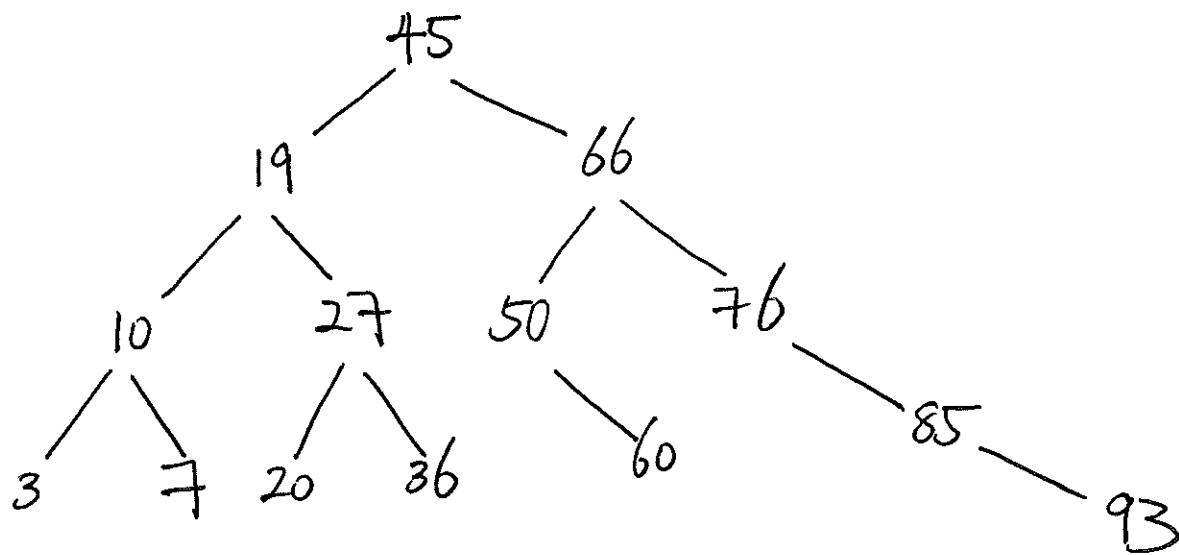
```
} else {
```

```
    return nullptr;
```

```
}
```

```
}
```

Search of BST



find(30)

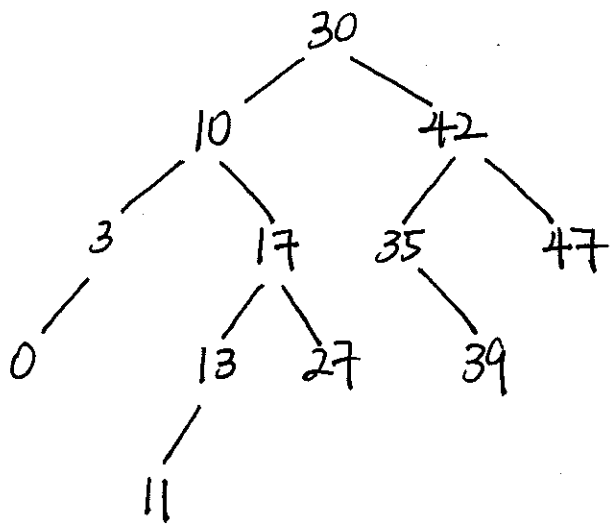
find(20)

Comparing with linear search

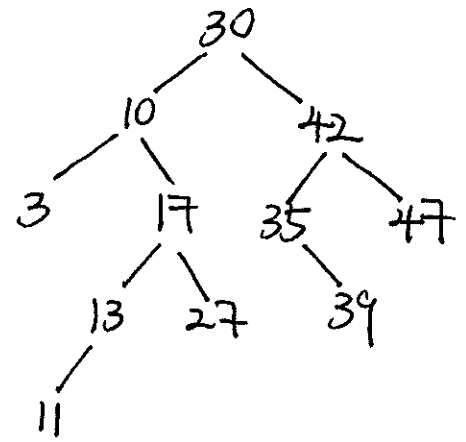
45 66 76 19 50 10 27 60 85 36 93 3 7 20

Binary Search Tree Deletion: delete a node n with key K

1° if n is a leaf

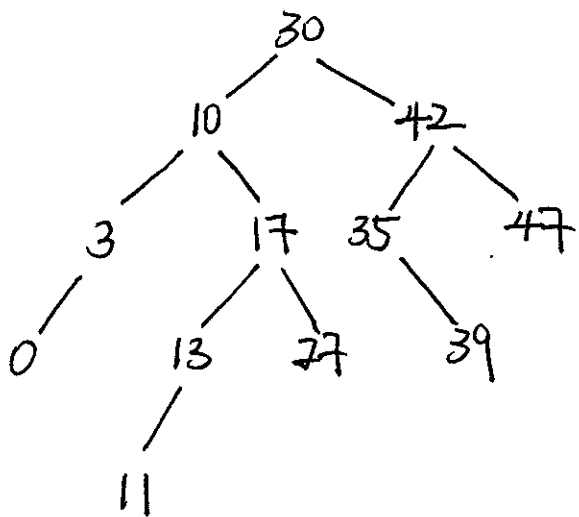


delete 0
→

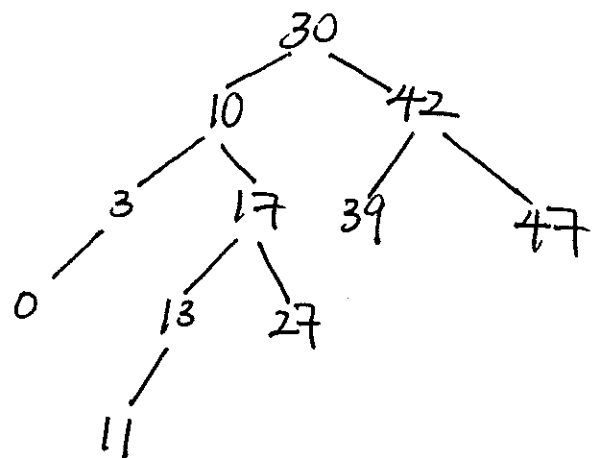


2° if n has only one child

$\text{child} \rightarrow \text{parent} = n \rightarrow \text{parent};$



delete 35
→



3° if n has two children

find the minimum on the $n \rightarrow \text{right}$ subtree

review every key on $n \rightarrow \text{right}$ subtree is $> k$

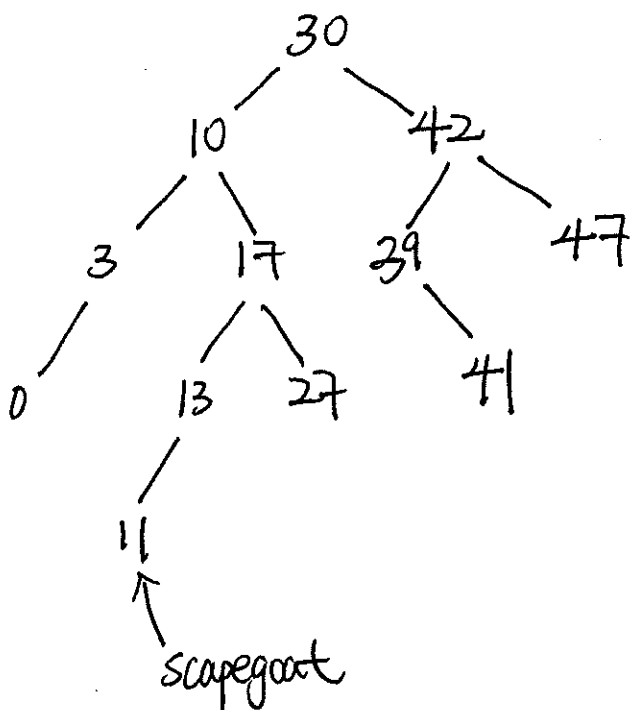
So min is the next node of n in ascending order

min: the left-most node of $n \rightarrow \text{right}$ subtree

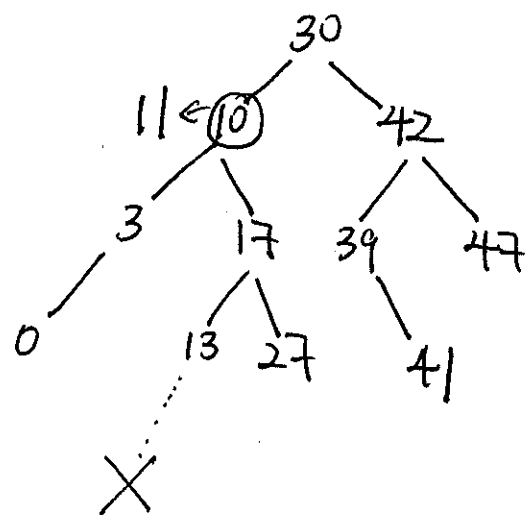
↑
scapegoat

$n \rightarrow \text{data} = \text{min} \rightarrow \text{data} :$

delete min from $n \rightarrow \text{right}$ (recursion)



delete 10
→



```
bool erase(T val, Node* t) {
```

```
    if (t == nullptr) return false;
```

```
    if (val < t->data)
```

```
        return erase(val, t->left);
```

```
    else if (val > t->data)
```

```
        return erase(val, t->right);
```

```
    else {
```

```
        if (t->left != nullptr && t->right != nullptr) {
```

```
            t->data = min(t->right)->data;
```

```
            return erase(t->data, t->right);
```

```
        }
```

```
        Node* child = t->left ? t->left : t->right;
```

```
        if (child) {
```

```
            child->parent = t->parent;
```

```
        }
```

```
        delete t;
```

```
        t = child;
```

```
        -- msize;
```

```
        return true;
```

```
}
```

Time Complexity :

insert

search : from root to a leaf (at most)

delete

What is defined like this ? (from root to a leaf)

$O(\text{Height of the tree})$

Worst Case : n nodes

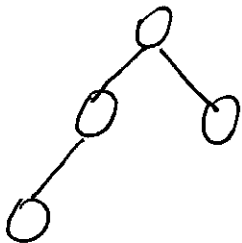


Height: $n-1$

$O(n)$

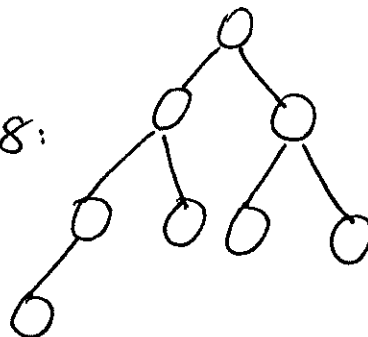
Best Case :

4:



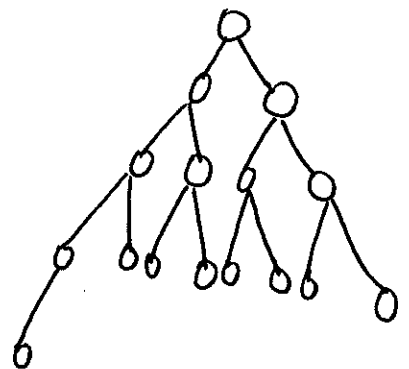
Height : 2

8:



Height : 3

16:



Height : 4

n nodes: Height: $\log_2 n$

$O(\log n)$

Implementing BST:

"dummy" header : m-header Node

m-header.~~right~~^{right} points to the smallest node

m-header.~~left~~^{left} points to the largest node

m-header.parent points to the root

the root's parent is m-header

How to check if a node is not root?

$n \rightarrow \text{parent} \rightarrow \text{parent} \underline{\neq} n$

begin(): return {m-header.right};

Implementing binary search tree

1. a struct/class representing BST node : Node

* Data Members

- data T
- left Node*
- right Node*
- parent Node*

* Constructors (using initializer list)

```
Node (const T& d , Node* l , Node* r , Node* p)  
: data(d) , left(l) , right(r) , parent(p)  
{  
}
```

If left , right , or parent is empty , it ~~shold~~ should
be set to nullptr .

↑
C++ keyword

2. A struct/class representing tree iterators : TreeIterator

* Data Member

- m_nodePtr Node*

* Member functions

- constructor

```
TreeIterator (Node* n) : m_nodePtr (n)
{
}
```

- operator*

return m_nodePtr → data ;

- Increment Operators

pre-increment

vs

post-increment

increment (m_nodePtr);

return *this;

// create a copy of *this

// increment (m_nodePtr)

// return the copy

How to increment $m_nodeptr$?

review the use of set iterators:

a set of values: 7 15 4 21 0

begin() ? 0

++ ? 4

(ascending order)

How to find the next smallest node after node n ?

1° if ($n \rightarrow \text{right}$) // if $n \rightarrow \text{right}$ subtree is not empty

// find the leftmost node of $n \rightarrow \text{right}$

$n = n \rightarrow \text{right};$

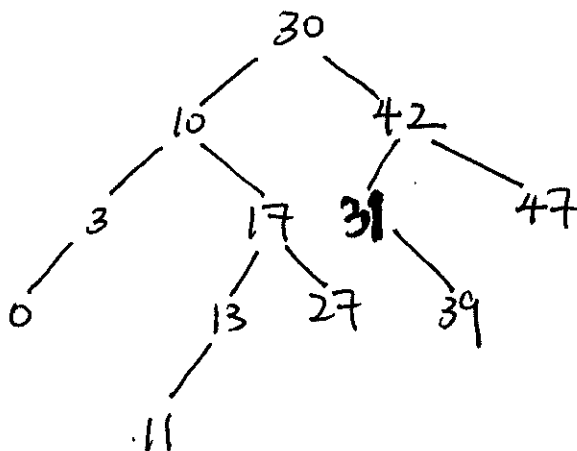
while ($n \rightarrow \text{left}$) {

$n = n \rightarrow \text{left};$

}

return n ;

e.g.



2° if $n \rightarrow \text{right}$ is empty

case a: n is a left child

then $n \rightarrow \text{parent}$ is the one

case b: n is a right child

while (n is its parent's right child) {

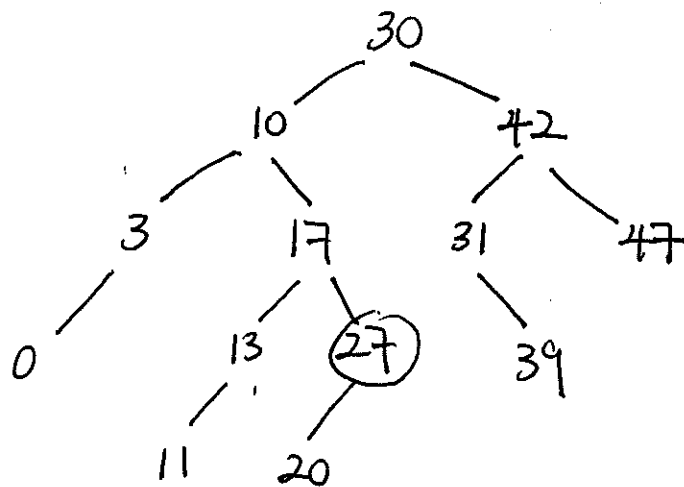
$n = n \rightarrow \text{parent}$

}

then $n \rightarrow \text{parent}$ is the one

&& n is not root

e.g.



case a: 13

case b: 27

47?

-Decrement operators

pre-decrement vs post-decrement

How to decrement $m_node \rightarrow r$?

Return in-order predecessor of n

1° if $(n \rightarrow \text{left})$

then return the right-most node of $n \rightarrow \text{left subtree}$

2° if $n \rightarrow \text{left}$ is empty

case a: if n is a right child

then return $n \rightarrow \text{parent}$

case b: if n is the left child of its parent

then ~~return~~

while (n is the left child) {
 $n = n \rightarrow \text{parent};$ $\&\& n$ is not root

}

then return $n \rightarrow \text{parent}$.

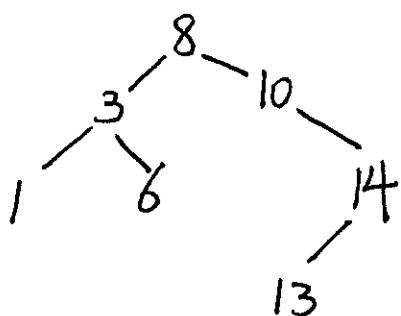
Review on Binary Search Tree

A BST is a binary tree, so each node has at most two children — left and right. Each node stores a key and optionally, an associated value.

Key property: For each node N in a BST,

keys in the left subtree of $N \leq$ the key in $N <$ keys in the right subtree of N

e.g. 8, 10, 3, 1, 6, 14, 13



review height:

The height of an empty tree: -1

For non-empty trees, the height of a tree is the number of edges of a longest path ~~between~~ from the root to a leaf.

What is the height of the tree? 3

If we want to search 13, we need to compare 13 with 8, then 10, then 14, then 13.

You can see when the height is higher, we may need more time to search a key. So we want the height of a BST to be as short as possible.

— A BST of n nodes

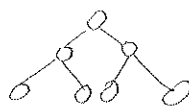
Best Case: Height = $O(\log n)$

Worst Case: Height = $O(n)$

e.g. 1, 2, 3, 4

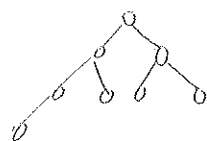
No difference with linear search

e.g. 7 nodes



$\log_2 7 = 2.8 \dots$

8 nodes



$\log_2 8 = 3$

We want the tree to be balanced, which means we have roughly equal # of nodes on the left & right.

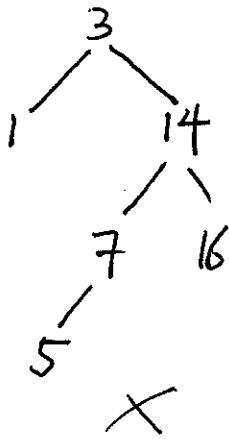
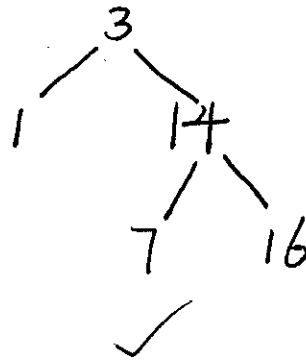
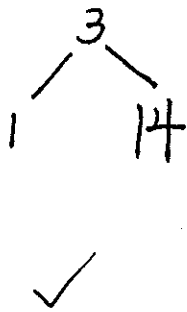
Balanced Binary Tree (Recursive Definition)

An empty tree is balanced.

A non-empty binary tree T is balanced if:

- (1) The left subtree of T is balanced, and
- (2) The right subtree of T is balanced, and
- (3) The heights of the left and right subtrees of T differ by at most 1.

e.g.



You can write

Our goal is to have a BST that is always balanced after any insertion or deletion.

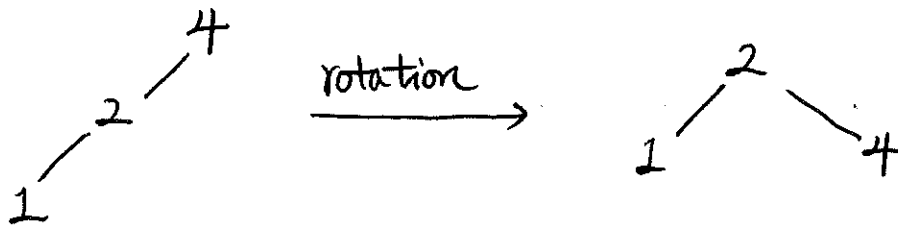
This is called

Self-balancing Binary Search Tree

The first self-balancing BST was invented in 1962 by Adelson-Velsky and Landis.

AVL Tree

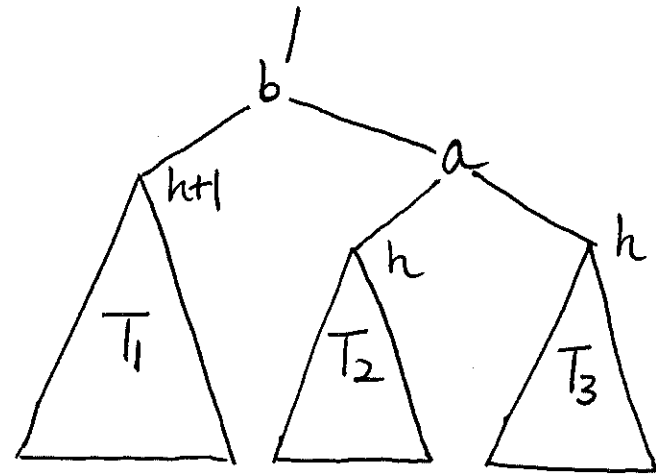
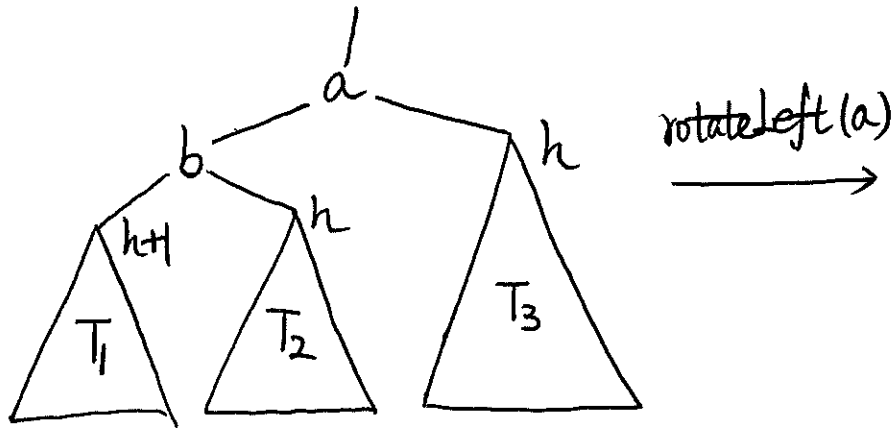
The core of an AVL tree is rotation. If a BST becomes unbalanced, rotate the tree to make it balanced again. e.g.



Is it a BST?
Is it balanced?

There are 4 cases when a BST is not balanced.

Case 1: Left-Left case



Is it a BST?
Is it balanced?

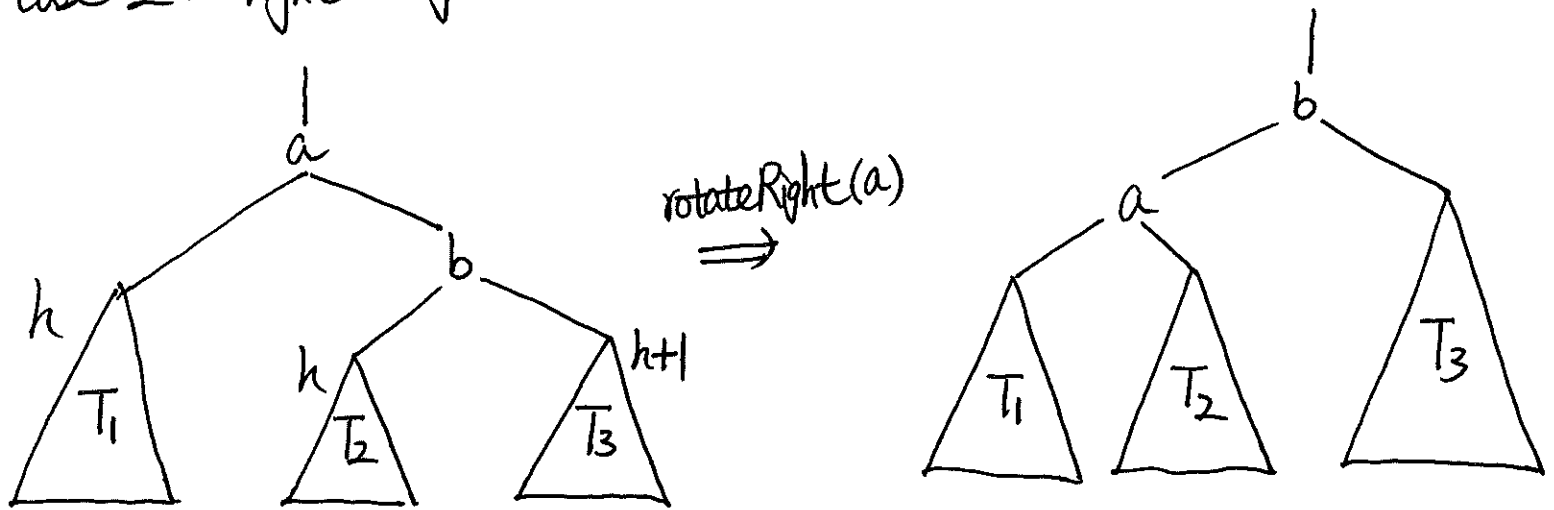
```

NodePtr rotateLeft(NodePtr a) {
    NodePtr b = a->left;
    a->left = b->right;
    if (b->right) b->right->parent = a;
    b->right = a;
    b->parent = a->parent;
    a->parent = b;
    a = b;
    return a;
}
  
```

Order is important
from the smaller tree a
to the bigger tree b

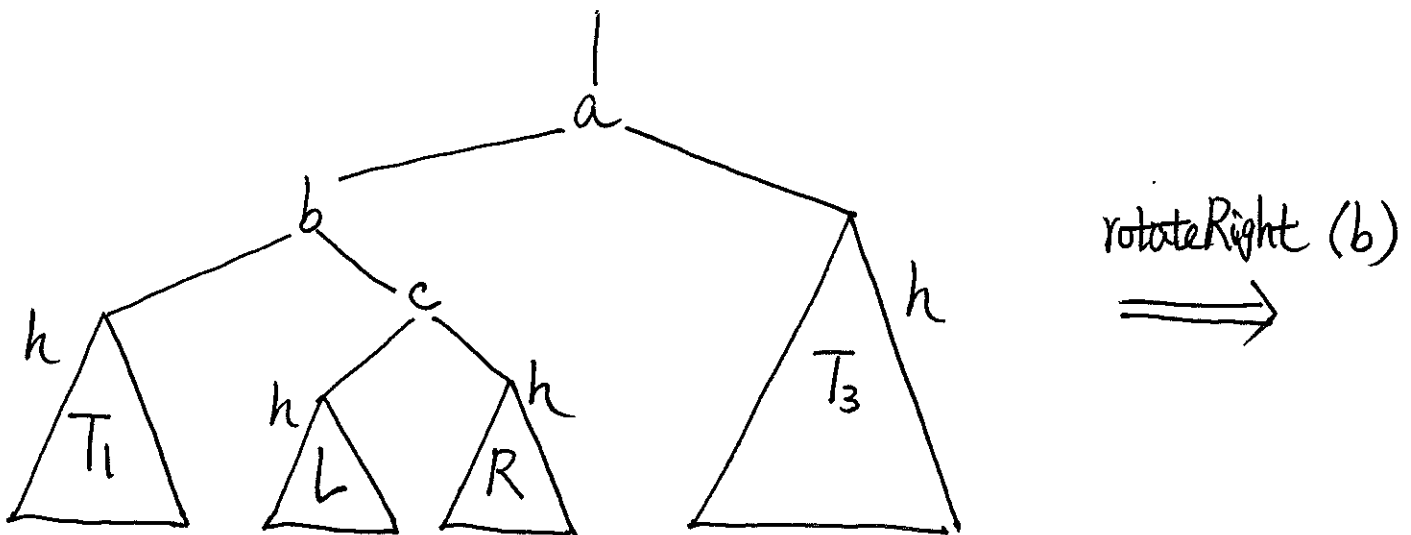
}

case 2: right-right case



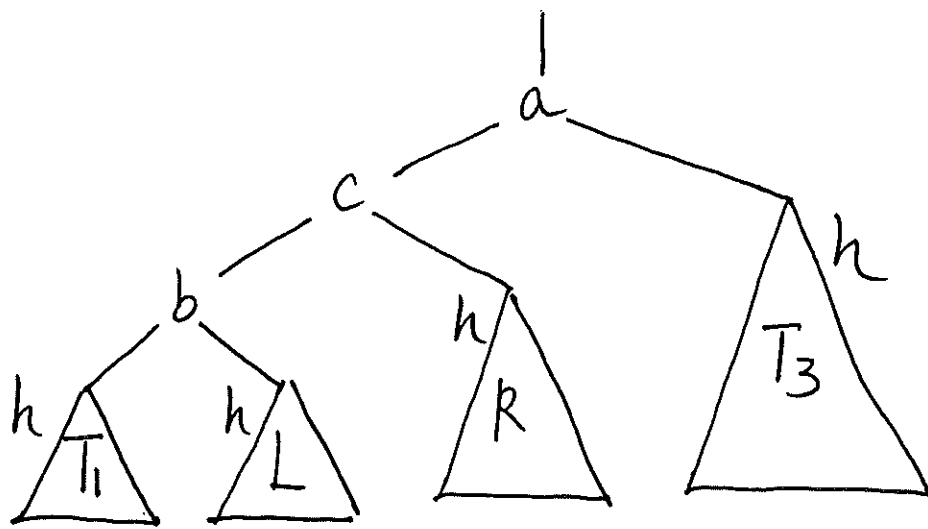
Leave the code to the readers.

case 3: left-Right case



Is it balanced? No

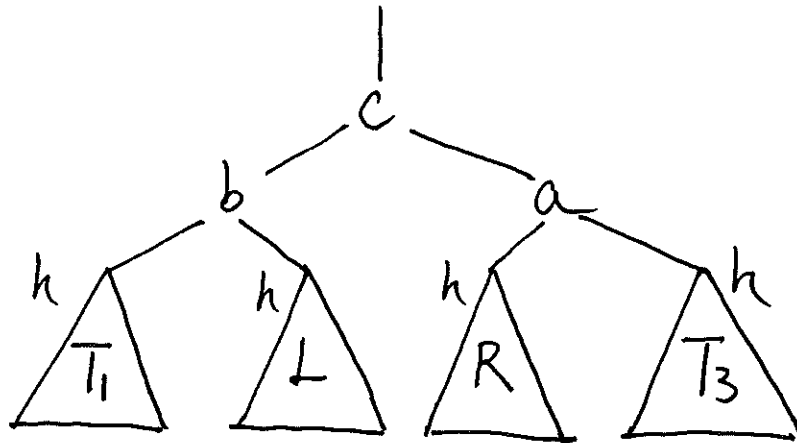
Can we use case 1 to handle this? No



$\text{rotateLeft}(a)$
 $\Rightarrow$

Is it a BST?

Is it balanced? Back to case 1: Left-Left



```
NodePtr doubleRotateLeft(NodePtr a) {
```

```
    NodePtr b = a->left;
```

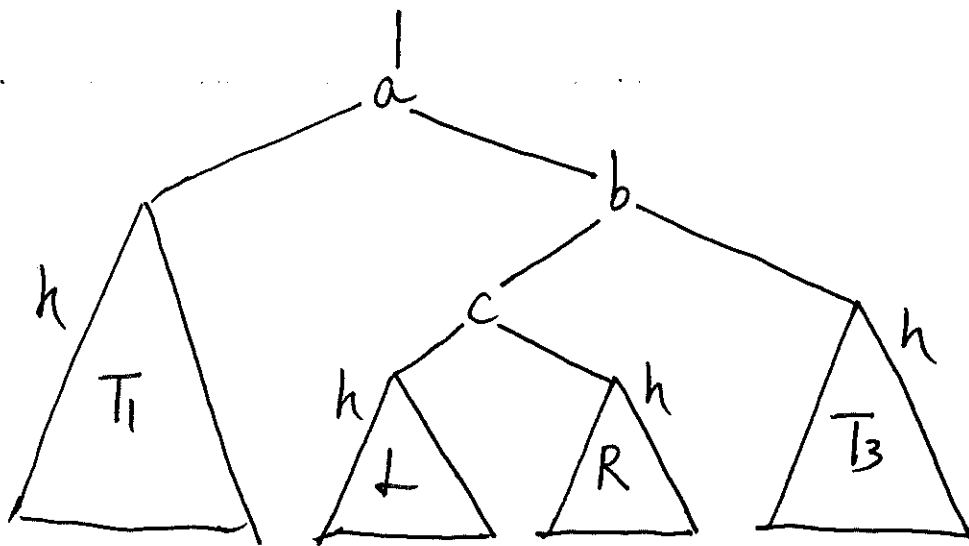
```
    NodePtr c = rotateRight(b);
```

```
    a->left = c;
```

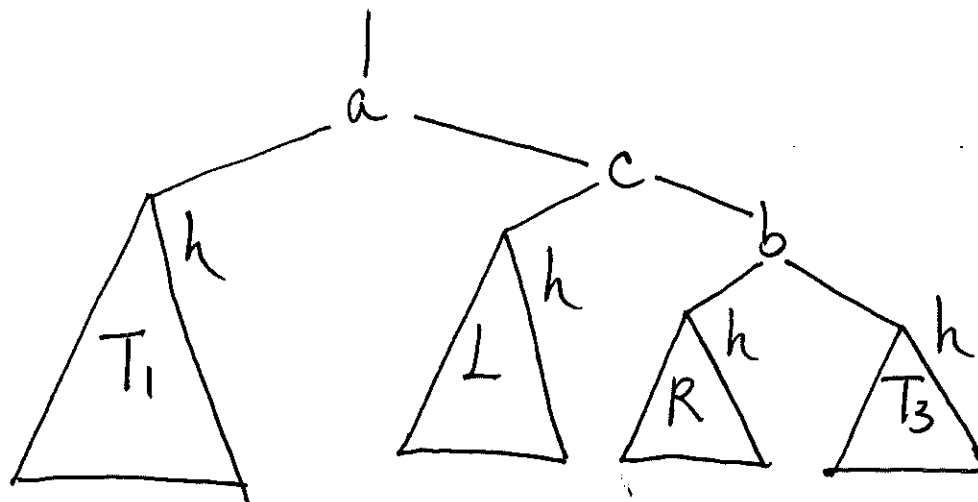
```
    return rotateLeft(a);
```

```
}
```

case 4: Right-Left case



$\text{rotateLeft}(b)$
 $\Rightarrow$



$\text{rotateRight}(a)$
 $\Rightarrow$

